Octopus — comandos e diagnóstico v1.4.3

As páginas web são **Status** (`/`) e **Configuração** (`/config`), ambas na porta 80. O endpoint `/status` continua sendo JSON para atualização e integrações. Na configuração, **Salvar e testar login** primeiro grava/verifica as prefs e então inicia teste assíncrono de autenticação, SELECT 1 e leitura das colunas de `public.netscan`. POST `/netscan/testar` inicia o teste sem nova gravação. O resultado aparece no painel e no log SD com data/hora. O teste não grava equipamentos nem comprova permissão de escrita; essa é validada no netscan. Durante o teste, mudanças de credenciais e início de netscan são recusados. Banco **LogsRede** e tabela **public.netscan** são fixos no firmware, sem campos para editar seus nomes. O servidor, porta, usuário e senha continuam configuráveis. A página usa formulários responsivos, cartões de seções e destaque de erros SQL.

Os novos registros de `/log.txt` recebem `[DD/MM/AAAA HH:MM:SS] [uptime=Ns]` no fuso configurado. Antes de o relógio estar válido, recebem `[SEM_NTP]` e uptime. Mensagens com várias linhas recebem o mesmo instante em cada linha. O formato também segue para `/log_old.txt` após rotação. Registros antigos não são reescritos; consulte com `sd_log` ou download pelo painel.

Netscan periódico e PostgreSQL

- `netscan_interval:15`: salva intervalo de 1 a 1440 minutos (padrão 15).
- `netscan_auto:on`: ativa a agenda persistente.
- `netscan_auto:off`: desativa a agenda, mantendo `netscan` manual disponível.
- `netscan_status`: informa agenda, intervalo, contagem e último resultado SQL.

A primeira execução automática ocorre após o intervalo contado do boot; salvar configuração reinicia essa contagem. Um disparo automático nunca sobrepõe uma varredura ativa. Falhas de pré-condição (Wi-Fi/SD) são registradas e a nova tentativa fica para o próximo intervalo. Não há repetição acumulada de disparos.

O painel permite configurar servidor PostgreSQL (padrão 192.168.0.20), porta (5432), banco, usuário, senha, envio habilitado e agenda, sem reiniciar. Senha em branco mantém a atual; há opção explícita para apagá-la. A senha não é devolvida no status nem registrada em log. As prefs guardam essa configuração.

Ao concluir uma varredura, o firmware salva o JSON e, se habilitado, envia os registros em uma transação ao `public.netscan`, usando `ON CONFLICT(mac)`. JSON e SQL usam MACs em minúsculas. Sem ping, `response_time` é SQL `NULL`. `data_leitura` usa o relógio do PostgreSQL com `CURRENT_TIMESTAMP`, conforme o comando solicitado. Credenciais são capturadas no início da varredura; alterações durante uma operação valem na próxima execução.

Falhas de conexão, autenticação, timeout ou SQL aparecem na Serial, `/log.txt` e no estado da coleta. Falha no banco não invalida o JSON. Não há fila de reenvio de varreduras antigas: a próxima coleta envia seus próprios resultados. Sem SD gravável, o erro só pode ser observado na Serial/estado em RAM. Veja POSTGRESQL.md e sql/netscan.sql.

Download e upload pelo painel

Na seção **Arquivos do cartão SD** do painel (porta 80), informe o caminho e clique em **BAIXAR ARQUIVO**, ou escolha um arquivo e clique em **ENVIAR PARA O SD**. Download: `GET /sd/download?arquivo=/netscan.json` (também aceita subdiretórios). Upload: `POST /sd/upload`, formulário

`multipart/form-data`, um arquivo no campo `arquivo`, salvo na raiz usando o nome enviado. SD ausente, diretório ou caminho inválido gera erro; download de arquivo inexistente retorna 404.

Os nomes aceitam letras ASCII, números, espaços, ponto, hífen e sublinhado; download aceita `/` para diretórios. Caminhos com `..`, nomes reservados ao upload ou terminados em `.tmp`/`.bak` são recusados. Upload usa `/.octopus-upload.tmp`, verifica o tamanho recebido e só depois troca o destino com backup `/nome.upload.bak`. Em falha de troca, tenta restaurar o anterior. Backup restante após interrupção pode ser recuperado pelo SD ou comandos UDP.

Upload é recusado enquanto `netscan` estiver ativo. Downloads de logs ficam limitados ao tamanho inicial, mesmo se o log continuar crescendo. A transferência usa blocos de 1024 bytes; envio sem progresso por 1500 ms é interrompido e o navegador pode detectar o tamanho incompleto. Upload/download compartilham o servidor web do painel; uma transferência ocupa o atendimento HTTP/UDP do rádio até terminar, mas cede CPU para OTA e demais tarefas. O watchdog continua ativo.

Descoberta da rede (`netscan`)

- `netscan`: inicia uma varredura IPv4 da sub-rede Wi-Fi em segundo plano.
- `netscan_status`: consulta andamento, IPs verificados, MACs encontrados e resultado da gravação.
- `sd_read:/netscan.json`: consulta o último resultado salvo no SD.

Requer Wi-Fi conectado e SD ativo. Usa a máscara configurada, exclui rede e broadcast e aceita até 1022 hosts (/22); redes maiores são recusadas explicitamente. Em /24, cada IP sem resposta pode consumir aproximadamente 300 ms, além do processamento. Uma execução por vez; outros comandos continuam disponíveis. O LED azul alterna a cada 250 ms enquanto a operação estiver em andamento, incluindo a gravação no SD. Ao terminar ou falhar, retorna ao estado manual anterior (ou ao último estado solicitado por comando durante a varredura).

O JSON raiz é um dicionário cuja chave é o MAC. Cada objeto contém `ip`, `mac`, `response_time_ms` (ping ICMP em milissegundos ou `null`), `ping_replied` e `discovery`: `icmp_arp`, `arp_cache` ou `self`. O próprio Octopus entra como `self` com tempo `null`. MACs repetidos geram uma entrada, preferindo IP com resposta ICMP.

ARP só descobre MACs do segmento local. Clientes isolados, equipamentos adormecidos e filtros de rede podem impedir a descoberta. `arp_cache` identifica candidatos presentes na tabela ARP após a tentativa, inclusive quem bloqueia ping: não confirma que responderam nesta execução e pode conter cache antigo. Assim, a listagem não garante encontrar todos os equipamentos nem comprova disponibilidade sem ping.

O arquivo é substituído após a varredura completa, usando `.tmp` e `.bak` para preservar o anterior em falhas. Mudança de IP/rede ou perda do Wi-Fi aborta a coleta. O último resultado fica disponível para leitura enquanto a próxima coleta acontece. Não há envio espontâneo ao último cliente UDP: consulte `netscan_status`.

Revisão documental: **31/10/2026**. Histórico local e catálogo de todas as placas.

O painel HTTP (porta 80) exibe hardware, firmware, memória, rede, relógio, armazenamento e diagnóstico de reinicialização. Também funciona no portal OCTOPUS_CONFIG_PORTAL, usando o endereço do ponto de acesso.

Catálogo de comandos UDP

Envie um comando textual por datagrama à porta configurada, padrão **4210**. Os nomes são normalizados para minúsculas; argumentos preservam a caixa. A Serial mostra logs, mas não há leitor de comandos Serial neste projeto. O limite interpretado é 255 bytes; datagramas maiores são **truncados** pelo receptor atual, portanto não envie comandos acima desse tamanho.

`help` retorna o cabeçalho `COMANDOS` seguido de um comando por linha. Quando aplicável, a sintaxe do argumento aparece após `:`; a lista não mistura descrições nem instruções de transporte aos comandos.

Área	Comandos
Sistema	`help` · `name` · `info` · `status` · `version` · `build` · `reason` · `crash_info` · `alive` · `reboot` · `reset_wifi` · `reset_counters` · `uptime`
Hardware	`temp` · `cpu` · `ram` · `heap` · `flash`
Rede	`net_info` · `mac` · `rssi` · `ip` · `wifi_list`
Relógio	`time` · `date`
Clima	`age`
LED	`led_on` · `led_off` · `led_blink[:ms]`
SD	`sd_status` · `sd_list` · `sd_log` · `sd_clear_log` · `sd_size:/arquivo` · `sd_read:/arquivo` · `sd_write:/arquivo:texto` · `sd_del:/arquivo`

`led_blink` alterna o LED no intervalo informado, padrão 1000 ms, mínimo 50 ms. `sd_list` lista a raiz; `sd_status` informa disponibilidade, não capacidade. `sd_read` e `sd_log` enviam o arquivo por UDP em sequência, entre marcadores de início/fim. Aguarde todas as respostas.

Exemplos de consulta:

```
info
net_info
reason
age
sd_list
sd_read:/udp_log.txt
```

Não há handlers `wifi_add`, `sd_append`, `sd_test`, `sd_mkdir` ou `sd_rmdir` nesta variante. Configure as redes no portal. Escrita/exclusão, reset de configuração, reinício e leitura/apagamento de crash têm efeitos reais; use somente rede confiável.

Prometheus e configuração persistente

`age` retorna a idade em segundos dos dados de Porto Alegre desde o último sucesso no Open-Meteo. Antes da primeira atualização, informa que o clima ainda não foi sincronizado. Após 30 minutos, indica DESATUALIZADOS; falhas posteriores não reiniciam a idade. É uma consulta ao cache, sem acesso à rede.

O painel permanece na porta 80. A aba **Configurações** → **Prometheus** permite ativar o endpoint `/metrics`, escolher uma porta TCP exclusiva (padrão 9120) e informar o IP/hostname do servidor autorizado a coletar. O

endereço vazio permite qualquer coletor da rede. A configuração fica nas prefs, é recuperada no boot e pode ser alterada sem reiniciar ou reenviar credenciais Wi-Fi. O Prometheus consulta o Octopus por pull; o dispositivo não envia texto de métricas à porta 9090 do servidor. Portas 80 e 3232 não podem ser usadas pelo endpoint de métricas.

As métricas incluem identificação, disponibilidade, Wi-Fi, CPU, FreeRTOS, heap, PSRAM, flash, GPIOs usados pelo firmware, requisições e diagnóstico SD. Temperatura e umidade de Porto Alegre usam Open-Meteo com os labels da Central; a idade é exportada em `esp_weather_data_age_seconds`. Energia, sensores ambientais locais, relés, ADC e MQTT sem integração são omitidos, como no ESP32C6. Uso de CPU requer duas amostras válidas de runtime stats do FreeRTOS. Consulte o catálogo e o exemplo de scrape em PROMETHEUS.md. Visão geral do firmware e hardware em README.md.

Todas as amostras Prometheus incluem `device="MAC_DO_OCTOPUS"`, inclusive as métricas SD. Reset usa valor 1 e código no label `reason`; IP inclui `interface`; GPIO usa `pin`; logs SD usam caminhos completos no label `file`. Contadores `application_*` são HTTP; comandos UDP usam `esp_udp_*`. Para diagnóstico do transporte, consulte `esp_udp_listener_up`, `esp_udp_packets_received_total`, `esp_udp_replies_sent_total` e `esp_udp_send_errors_total`. Pacotes são contados antes da fila; respostas exigem envio completo aceito pela pilha local. Os counters reiniciam no boot. `sdcard_log_records_total` e `sdcard_log_errors_total` atendem aos painéis Log Records e Log Error Records, contando gravações e falhas de registro.

`reset_wifi` apaga rede e relógio, mantendo a configuração Prometheus e os contadores de boot/reset/watchdog persistentes. `reset_counters` zera somente as chaves de contadores no namespace `metric_counts`, preservando a configuração Prometheus. `reboot` reinicia o dispositivo. `version` e o painel mostram a versão v1.3.3.

Em resets por watchdog, `crash_info` também mostra a última etapa do rádio preservada em RTC pela v1.3.3. O resumo de `IDLE1` sozinho não indica onde o rádio travou. Logs que não obtêm o mutex em até 100 ms são descartados e contados em `sdcard_log_errors_total`, sem prender a recepção indefinidamente.

Diagnóstico de falhas

Envie `crash_info` à porta UDP configurada (padrão 4210). O comportamento segue a Central: consulta o core dump ELF em flash, mostra panic, tarefa, PC, backtrace e SHA256 do ELF e arquiva o relatório no SD como `/crash_YYYYMMDD_HHMMSS.txt` (ou `SEM_NTP` com uptime). Só apaga o core dump após reabrir e conferir integralmente o arquivo. Sem SD ou com falha de gravação, preserva o registro interno. Sem suporte a core dump no firmware, informa explicitamente essa condição. A captura requer core dump ELF em flash habilitado e partição coredump. `reason` consulta o motivo da última reinicialização sem apagar registros.

Histórico de comandos e respostas

`/udp_log.txt` registra cada comando UDP aceito pelo receptor (RX) e cada tentativa de resposta (TX), incluindo conteúdo, data/hora local, uptime, IP/porta do cliente, tamanho e resultado do envio. ENVIADO significa que a pilha UDP aceitou o pacote; UDP não confirma a entrega ao cliente. Antes de sincronizar o relógio, use o uptime para ordenar os eventos. Falhas por fila cheia são registradas. O interpretador aceita até 255 bytes por comando; datagramas maiores são truncados ao limite do receptor.

Ao atingir 512 KiB, o arquivo é movido para `/udp_log_old.txt`, substituindo o histórico anterior. A gravação é serializada entre as tarefas. Sem cartão SD, os comandos continuam funcionando sem persistir histórico. O

log geral do sistema continua em `/log.txt`.

- `sd_list`: lista os arquivos e relatórios.
- `sd_read:/udp_log.txt`: consulta o histórico atual.
- `sd_read:/udp_log_old.txt`: consulta a rotação anterior.
- `sd_read:/crash_YYYYMMDD_HHMMSS.txt`: consulta um relatório arquivado.

A leitura de arquivos limita-se ao tamanho inicial para não continuar indefinidamente ao registrar as próprias respostas da leitura do log.